Department of Computer Science and Engineering

CSE472 - Web and Internet Programming Lab

Project Report

LocationKhuji: An AI-Powered Bangladesh Location and Service Discovery Platform

Field	Information
Student Name	Md. Fardin Hossain
Student ID	2023100000569
Course	CSE472.1 - Web and Internet Programming Lab
GitHub	**Github_LocationKhuji**
Live Website	**https://locationkhuji.vercel.app/**
Submission Date	09 June 2026

Submitted to:

Abid Ahmad

Lecturer, Department of Computer Science and Engineering

Southeast University

Abstract

LocationKhuji is a full-stack, map-first web application designed to improve local place discovery in Bangladesh. The platform allows users to search for rental flats, pharmacies, hospitals, restaurants, and local service providers using both structured filters and natural-language AI search. The system combines a React single-page frontend, an Express.js API gateway, MongoDB Atlas geospatial storage, Firebase Authentication, Cloudinary media hosting, Socket.io real-time updates, and external location services such as OpenStreetMap Nominatim and Overpass.

This improved report presents the project through a standard Software Development Life Cycle structure: planning, feasibility, requirement analysis, design, implementation, testing, deployment, maintenance, risks, and future enhancement.

Table of Contents

1. Introduction

1.1 Background

Bangladesh has many small businesses, clinics, pharmacies, food places, rental properties, and service providers that are not always easy to discover through global map applications or scattered social media posts. Users need local context, Bengali or mixed-language search support, accurate area matching, and reliable map boundaries. LocationKhuji addresses this by focusing only on Bangladesh and combining geospatial search with AI-assisted query understanding.

1.2 Problem Statement

The main problem is the absence of a unified Bangladesh-focused platform where people can discover nearby essential services, rental places, and businesses using natural language. Existing information is often scattered, outdated, difficult to filter, or not connected to precise coordinates.

1.3 Objectives

- Build a responsive full-stack web application for Bangladesh-focused location discovery.
- Support AI-powered natural-language search for English, Bengali, and mixed Banglish queries.
- Provide keyword, category, radius, and location-based search over MongoDB geospatial data.
- Allow owners to create, update, and manage listings with images and map coordinates.
- Provide reviews, ratings, saved listings, reporting, and admin moderation features.
- Use real-time updates so newly created or seeded listings appear without manual refresh.
- Document the system using SDLC phases so it can be maintained after delivery.

1.4 Scope

In Scope	Out of Scope
Web frontend, backend API, database schemas, authentication, image upload, search, map view, dashboards, testing, deployment and maintenance documentation.	Native Android/iOS app, payment system, advanced analytics billing, offline mobile maps, and distributed microservices.
Five listing categories: flat, pharmacy, hospital, restaurant and service.	International discovery outside Bangladesh and turn-by-turn navigation.

1.5 Stakeholders

Stakeholder	Need
General users	Find nearby flats, pharmacies, hospitals, restaurants and services quickly.
Business owners	Publish and manage listings with images, contact details and map location.
Administrators	Monitor users, listings, featured status, reports and platform statistics.
Future maintainer	Understand architecture, configuration, risks, tests and maintenance tasks.

2. SDLC Methodology and Project Planning

2.1 Selected Model

The project followed an iterative and incremental SDLC model. This was suitable because authentication, map search, AI search, listing CRUD, dashboards, file upload, and real-time updates could be designed, implemented, tested, and refined in increments.

2.2 SDLC Phase Mapping

Phase	Project Activities	Deliverables
Planning	Problem identification, scope definition, technology selection, timeline preparation.	Project proposal, feature list, schedule.
Requirement Analysis	Identify user roles, functional requirements, non-functional requirements and constraints.	Requirement tables, use cases, access matrix.
Design	Design architecture, database schemas, API routes, UI flow and search data flow.	Architecture diagram, ERD, API catalog.
Implementation	Build React frontend, Express API, MongoDB models, Firebase auth, Cloudinary upload, AI and OSM pipelines.	Working monorepo and deployed site.
Testing	Manual functional tests, role tests, search tests, upload tests and responsive checks.	Test cases and traceability matrix.
Deployment	Configure hosting, environment variables, database and external services.	Live frontend, backend API and cloud database.
Maintenance	Plan monitoring, secret rotation, backups, bug fixing and future enhancements.	Maintenance plan and risk register.

2.3 Timeline

Phase	Duration	Activities
Planning and Design	Week 1 (May 10–13)	Requirements, UI planning, technology selection and schema/API design.
Backend Development	Weeks 2-3 (May 14–26)	Express setup, MongoDB models, Firebase auth, listing APIs, AI search and OSM integration.
Frontend Development	Weeks 3-5 (May 17–27)	React routes, MapPage, ListPage, dashboards, listing form, responsive UI and state stores.
Integration	Week 4 (May 22–28)	MongoDB Atlas, Cloudinary upload, Socket.io events and external APIs.
Testing	Weeks 5-6 (May 28–Jun 1)	Functional, Bengali/Banglish search, responsive and bug-fix testing.

Documentation	Week 6 (Jun 1–5)	README, technical report, SDLC report improvement and final deployment review.

3. Feasibility Study

Area	Assessment
Technical	High. The stack is widely supported: React, Express, MongoDB Atlas, Firebase, Cloudinary, Leaflet and Socket.io.
Operational	High. The roles match real operation: users search, owners manage listings, admins moderate.
Economic	Moderate to high. Free or low-cost tiers support a student project; production growth needs paid capacity.
Schedule	Moderate. The system has many modules, so iterative development and prioritization were required.
Maintenance	High if secrets, dependencies, indexes, API quotas and data quality checks are managed regularly.

4. Requirement Analysis

4.1 Functional Requirements

ID	Requirement
FR-01	User registration, login, logout, profile retrieval and email verification.
FR-02	Role-based access for user, owner and admin routes.
FR-03	Search listings by keyword, category, location, radius and map center.
FR-04	Parse AI searches into category, keywords, price, bedrooms, emergency flag and location.
FR-05	Display listings on an interactive map with markers, clustering, user location and radius circle.
FR-06	Allow owners/admins to create, edit, soft-delete and view listings.
FR-07	Upload listing images through the backend and store media metadata.
FR-08	Allow users to save listings and view saved listings.
FR-09	Allow users to submit/delete reviews and update average rating automatically.
FR-10	Allow users to report suspicious listings and allow admins to moderate data.
FR-11	Broadcast newly created or seeded listings through Socket.io.
FR-12	Support English and Bengali interface text through i18next localization.

4.2 Non-Functional Requirements

ID	Requirement	Implementation Approach
NFR-01	Usability	Responsive UI, touch-friendly map controls, simple dashboards.
NFR-02	Performance	MongoDB 2dsphere index, text search priority, caching and result limits.
NFR-03	Reliability	Groq to OpenRouter to regex fallback; OSM fetch runs in background.
NFR-04	Security	Firebase token verification, role middleware, cookies, upload limits and coordinate validation.
NFR-05	Maintainability	Monorepo workspaces, shared location config and documented module responsibilities.
NFR-06	Localization	English/Bengali translation files and Bengali digit normalization.

4.3 Roles and Access Matrix

Feature	User	Owner	Admin
Home page	Yes	Yes	Yes
Map/list/detail pages	Yes	Yes	Yes
Save listings	Yes	Yes	Yes
Create listing	No	Yes	Yes
Edit own listing	No	Yes	Yes
Manage users/listings	No	No	Yes

4.4 Main Use Cases

Use Case	Actor	Normal Flow
Search for a place	User	Enter query or filters -> backend resolves intent/location -> listings returned -> map/list updates.
Create listing	Owner	Enter details -> set coordinates -> upload images -> submit -> listing saved and broadcast.
Review listing	User	Open listing -> submit rating/comment -> review stored -> rating recomputed.
Moderate platform	Admin	View stats/users/listings -> update user status/role or listing featured status.
Recover password	User	Request reset -> receive code -> submit code and new password -> Firebase password updated.

5. System Architecture

5.1 High-Level Architecture

LocationKhuji uses a three-tier client-server architecture. The React frontend handles UI and map interaction. The Express backend owns business logic, authentication verification, search processing, uploads, and real-time events. MongoDB Atlas stores users, listings, reviews, files, reports and password-reset codes.

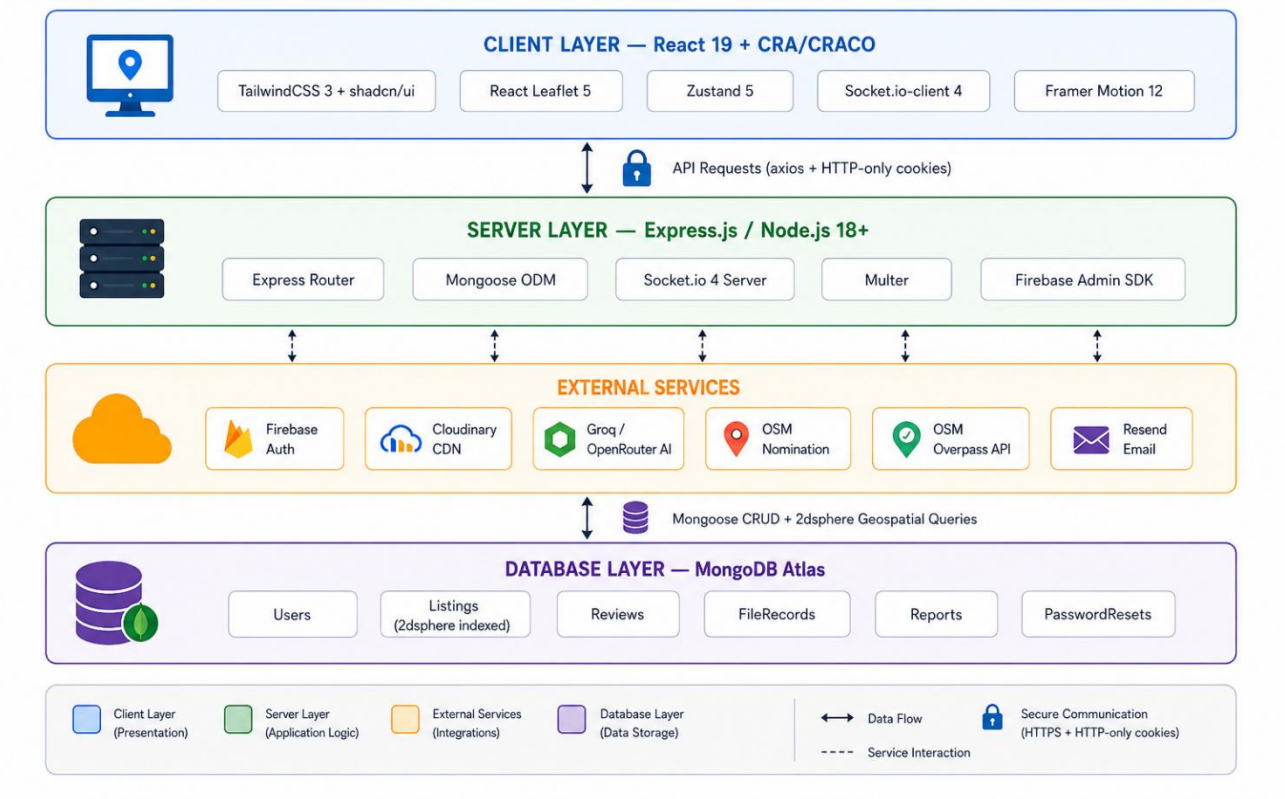

Figure 1: Three-tier architecture and external service integration

5.2 Search and Data Flow

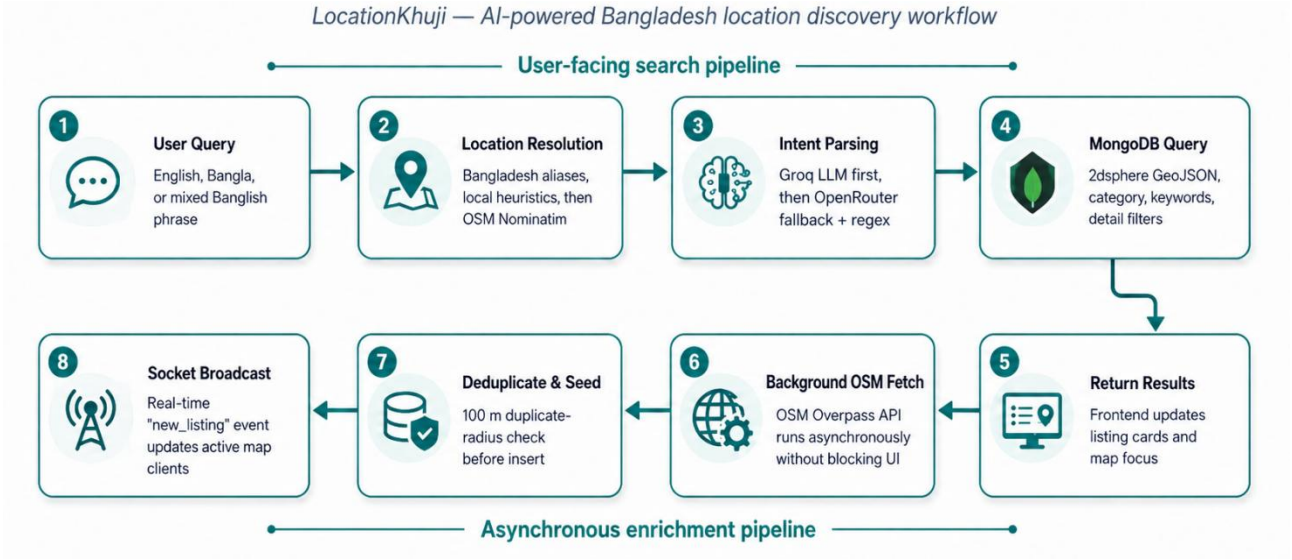

Figure 2: AI search and live OSM seeding flow

The search pipeline separates fast user response from slower background enrichment. AI search returns MongoDB results first, then optionally fetches OpenStreetMap data asynchronously and broadcasts newly seeded listings.

5.3 API Endpoint Summary

Group	Main Endpoints
Authentication	POST /api/auth/register, login, logout, refresh, forgot-password, reset-password; GET /api/auth/me.
Listings	POST /api/listings; GET nearby/search/my/detail; POST ai-search; PUT/DELETE /api/listings/:lid.
User actions	POST save/report; GET saved listings; PUT user profile.
Reviews	GET/POST listing reviews; DELETE review.
Files	POST /api/uploads; GET /api/files/:fid.
Admin	GET stats/users/listings; PUT user role/status; PUT listing feature.

5.4 Security Design

- Firebase Admin SDK verifies ID tokens for protected endpoints.
- Role middleware restricts owner and admin actions.
- Listing creation validates required fields, category and Bangladesh coordinate boundaries.
- Multer limits upload size to 5 MB and accepts image file types only.
- Search regex input is escaped before MongoDB filtering.
- Production maintenance should pin CORS_ORIGIN and rotate exposed keys.

6. Database Design

MongoDB was selected because listings need flexible category-specific fields and efficient geospatial queries. Mongoose defines models, constraints and indexes.

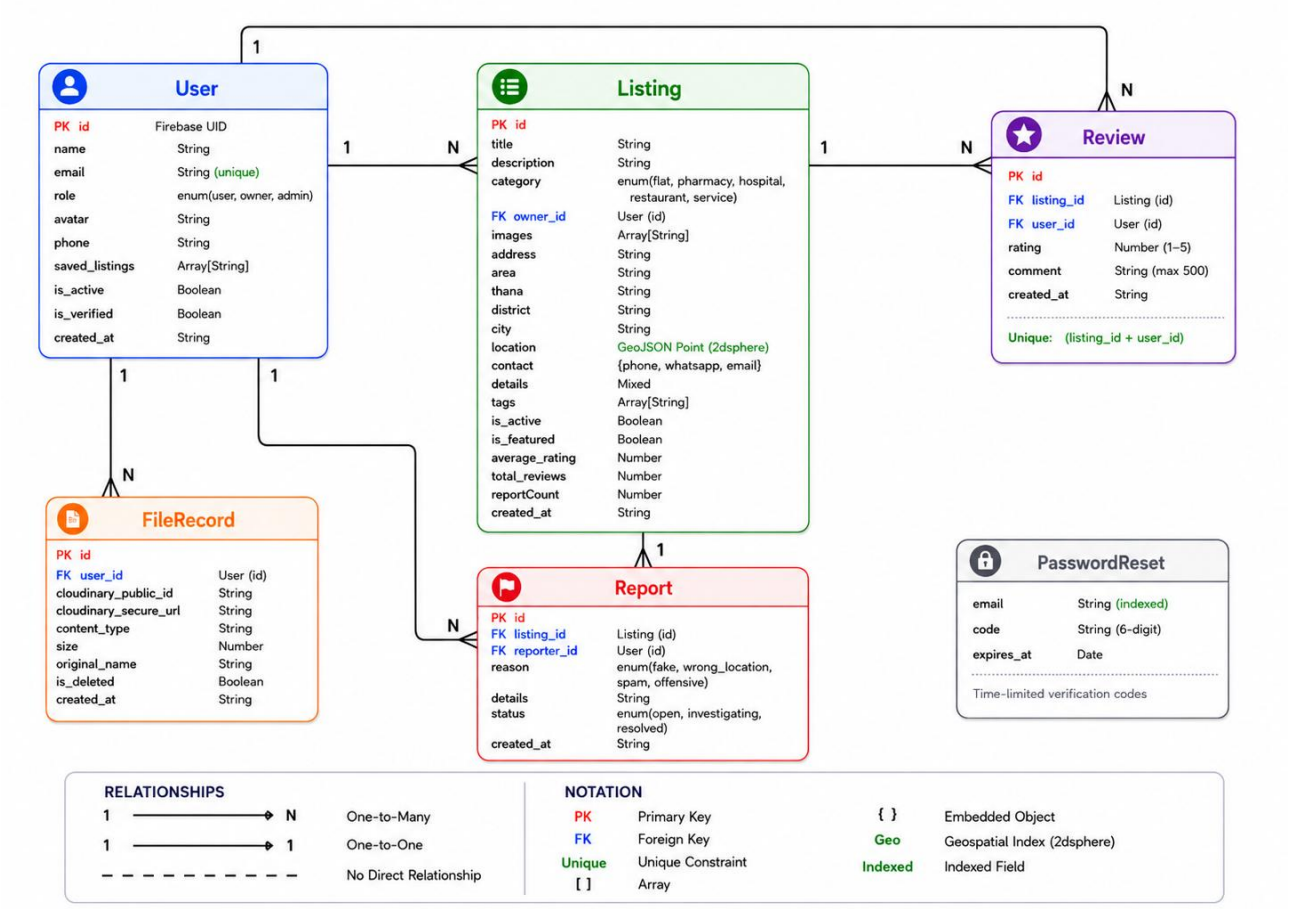

Figure 3: Collection relationship overview

6.1 Schema Summary

Collection	Important Fields	Indexes / Constraints
User	id, name, email, role, avatar, phone, saved_listings, is_active, is_verified	Unique id and email; role enum.
Listing	title, category, owner_id, images, address, area, thana, district, division, location, contact, details, tags, ratings	2dsphere location index; text index on title/area/thana; category enum.
Review	listing_id, user_id, rating, comment	Unique listing_id + user_id; rating 1-5.
FileRecord	user_id, cloudinary_public_id, secure_url, content_type, size	Unique id; upload metadata.
Report	listing_id, reporter_id, reason, status	Reason/status enums for moderation workflow.
PasswordReset	email, code, expires_at	Email index for reset-code lookup.

6.2 Design Decisions

- Listing.location is stored as a GeoJSON Point using [longitude, latitude].
- Category-specific attributes are stored in the details object for flexibility.
- Listings are soft-deleted using is_active instead of physical deletion.
- Review aggregation updates average_rating and total_reviews after changes.
- Saved listings are stored as listing IDs in the User collection.

7. Implementation Details

7.1 Technology Stack

Layer	Technologies	Purpose
Frontend	React 19, Router 7, TailwindCSS, shadcn/Radix, Framer Motion, Zustand, Axios	Responsive SPA, routing, UI, state and API requests.
Map	Leaflet, React Leaflet, react-leaflet-cluster	Interactive map, markers, clusters, radius and boundary overlay.
Backend	Node.js 18+, Express 4, Mongoose 8, Socket.io 4, Multer	REST API, DB access, real-time events and upload handling.
Services	MongoDB Atlas, Firebase, Cloudinary, Groq, OpenRouter, Nominatim, Overpass, Resend	Persistence, auth, media, AI, geocoding, public data and email.
Monorepo	npm workspaces	Separate frontend/backend apps and shared Bangladesh location utilities.

7.2 Monorepo Structure

```
LocationKhuji/
  package.json
  apps/backend-api-gateway/src/server.js
  apps/backend-api-gateway/src/locationResolver.js
  apps/frontend-search-hub/src/App.js
  apps/frontend-search-hub/src/pages/
  apps/frontend-search-hub/src/components/
  apps/frontend-search-hub/src/store/index.js
  packages/shared-config/bd-location-engine.js
```

Appendix A: Detailed Route Inventory

Area	Routes
Auth	/api/auth/register, login, logout, refresh, me, verify-me-dev, resend-verification, forgot-password, reset-password.
Listings	/api/listings, nearby, search, ai-search, my, :lid, :lid/save, :lid/report.
Reviews	/api/listings/:lid/reviews, /api/reviews/:rid.
Users	/api/users/me/saved, /api/users/me.
Files	/api/uploads, /api/files/:fid.
Admin	/api/admin/stats, users, users/:uid, listings, listings/:lid/feature.
Health	/api/health.

Appendix B: Source Files Reviewed

Module	Files
Backend API gateway	apps/backend-api-gateway/src/server.js
Location resolver	apps/backend-api-gateway/src/locationResolver.js
Frontend routes	apps/frontend-search-hub/src/App.js
API client	apps/frontend-search-hub/src/lib/api.js
State stores	apps/frontend-search-hub/src/store/index.js
Map workflow	apps/frontend-search-hub/src/pages/MapPage.jsx and components/MapView.jsx
Dashboards/listing form	apps/frontend-search-hub/src/pages/DashboardPages.jsx
Shared BD data	packages/shared-config/bd-location-engine.js and data JSON files

7.3 Feature Documentation and Screenshot

This section describes the ten primary features of LocationKhuji, each accompanied by a screenshot placeholder for visual documentation. The features span the complete user journey from discovery to interaction.

7.3.1 Home Page and Hero Search

The home page introduces LocationKhuji as a Bangladesh-focused location discovery platform. It contains a clean hero section, category shortcuts, popular area options, and a search bar that takes users directly to the map search experience.

Benefit: A new user can quickly understand the purpose of the system and start searching without navigating through multiple pages.

Screenshot:

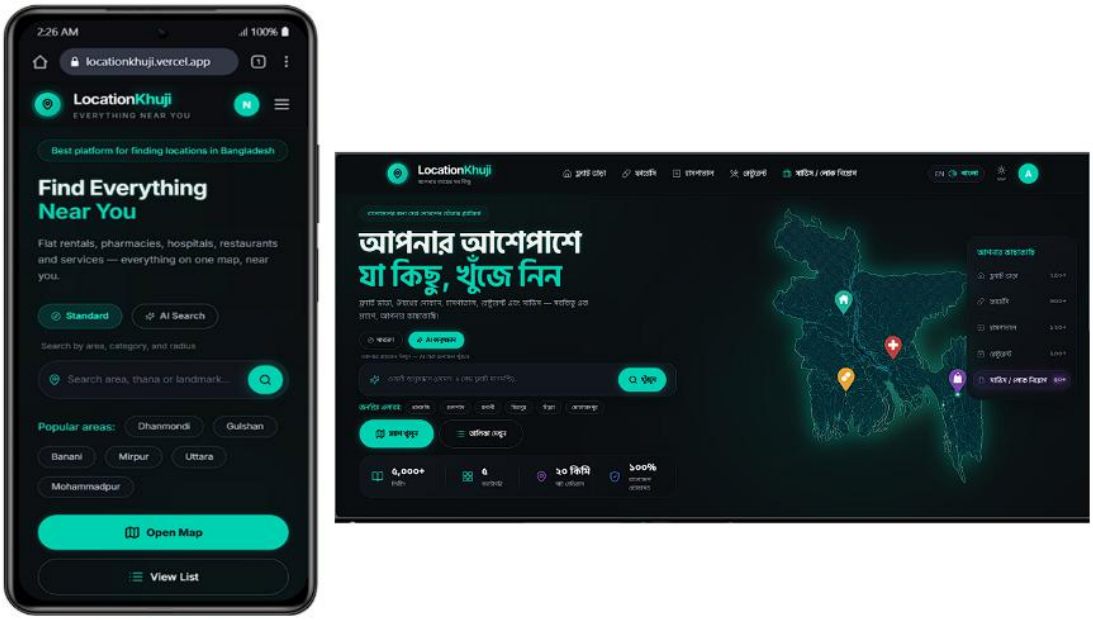

Figure 4.1: Home page and quick search interface of LocationKhuji.

7.3.2 User Registration and Login

LocationKhuji provides account registration and login through Firebase Authentication. During registration, users can select their role as a general user or business owner. The system also supports email verification and password reset.

Benefit: Users can securely access personalized features such as saved listings, reviews, dashboards, and listing management.

Screenshot:

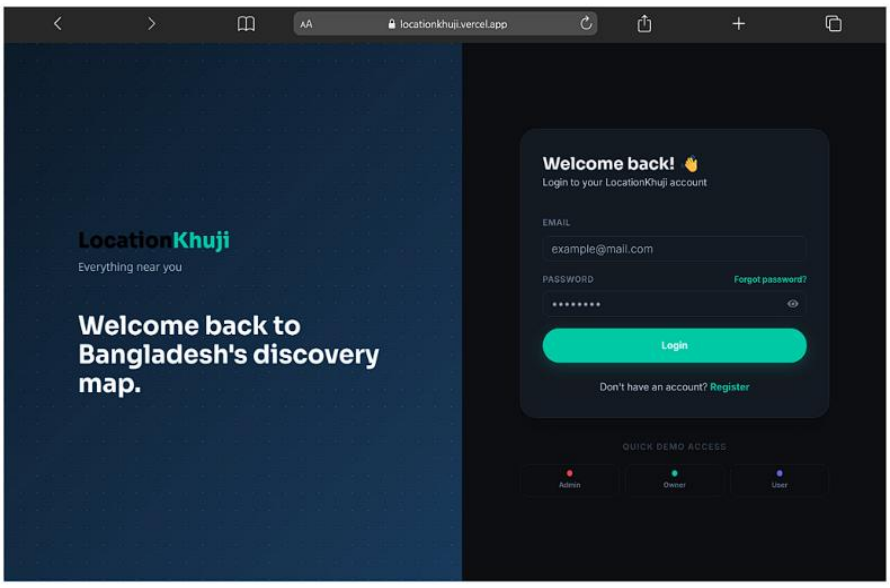

Figure 4.2: User authentication and role selection interface.

7.3.3 Role-Based Access Control

The platform has three main roles: user, owner, and admin. General users can search, save, review, and report listings. Owners can create and manage their own listings. Admins can manage users, listings, and platform statistics.

Benefit: Each user only sees the tools that match their responsibility, which keeps the system organized and secure.

Screenshot:

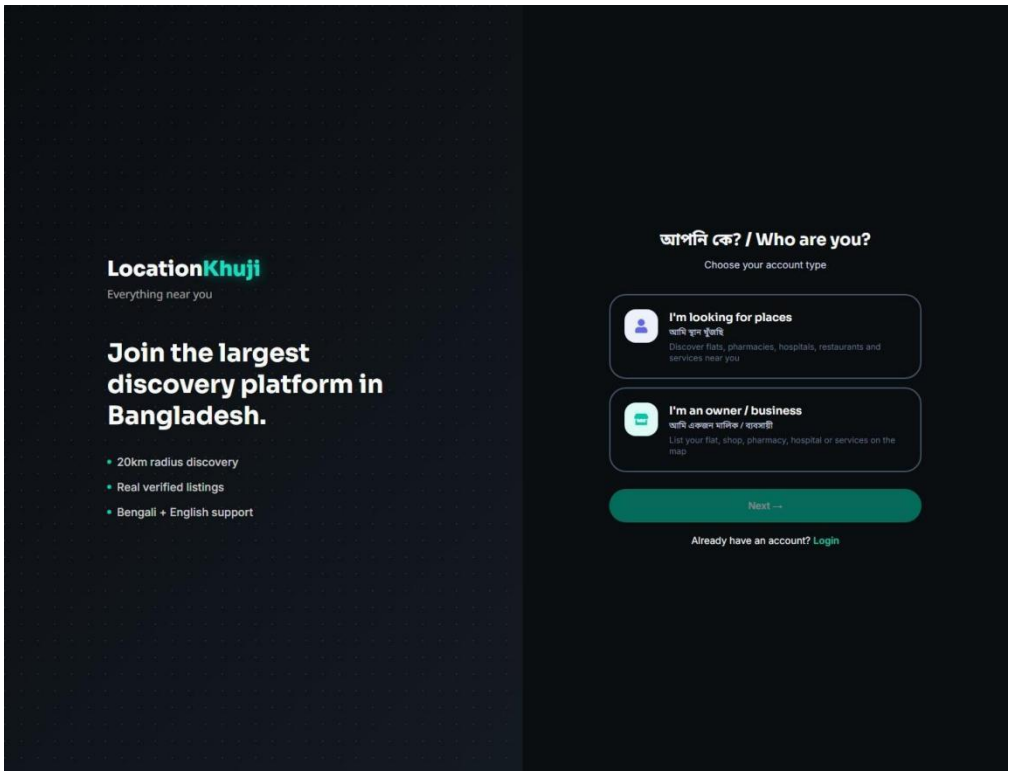

Figure 4.3: Role-based access control in LocationKhuji.

7.3.4 Interactive Map View

The map page is the main discovery interface of the application. It shows listings as markers on a Leaflet map, supports marker clustering, displays a selected search radius, and keeps the map focused inside Bangladesh.

Benefit: Users can visually explore nearby places and understand exactly where each listing is located.

Screenshot:

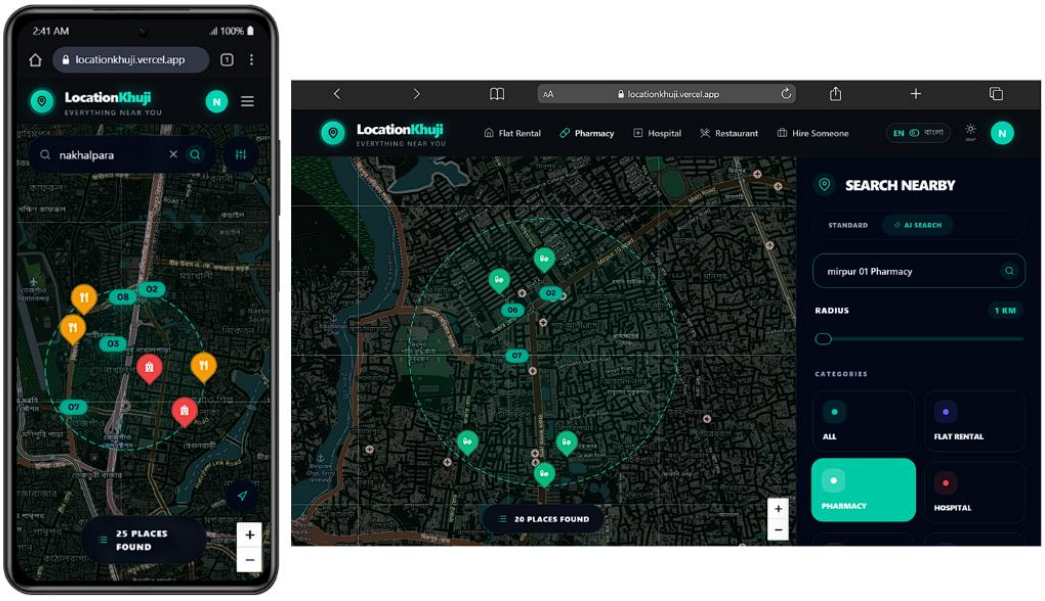

Figure 4.4: Interactive map view with location markers and search radius.

7.3.5 AI-Powered Smart Search

The AI search feature allows users to type natural language queries such as "pharmacy near Mirpur 10" or "2 bedroom flat in Dhanmondi under 20000". The backend extracts category, location, keywords, price, emergency need, and bedroom count from the query.

The system uses Groq as the primary AI parser, OpenRouter as a fallback, and a local regex-based parser if external AI services are unavailable.

Benefit: Users do not need to fill many filters manually; they can search in a natural and simple way.

Screenshot:

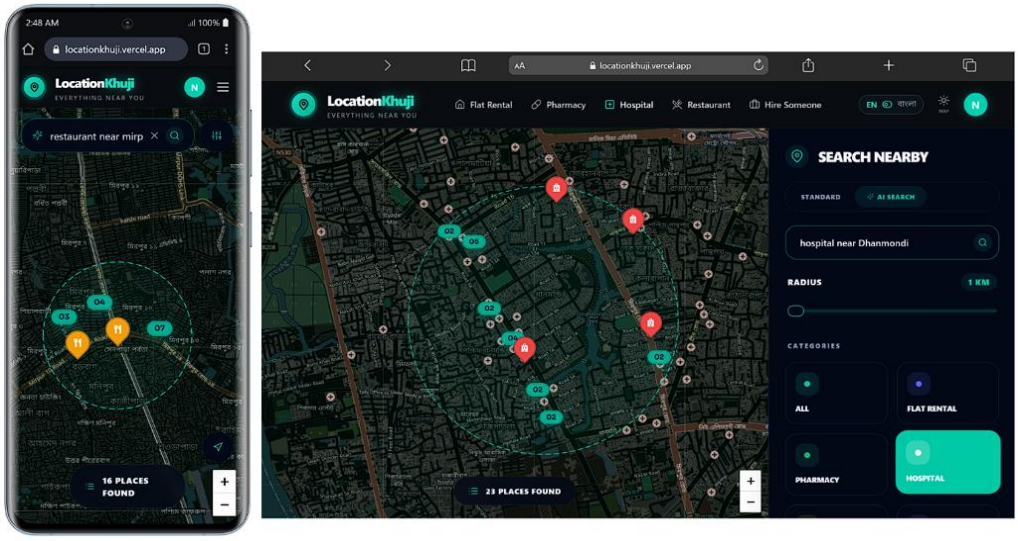

Figure 4.5: AI-powered natural language search result.

7.3.6 Standard Search and Filters

Keyword-based search with a category, radius slider (1–20 km), and location-aware results sorting. The backend resolves location names into geographic coordinates through a three-tier fallback system: hardcoded area dictionary, preposition pattern extraction, and Nominatim geocoding.

Benefit: Users who prefer search with a category and location-aware results sorting.

Screenshot:

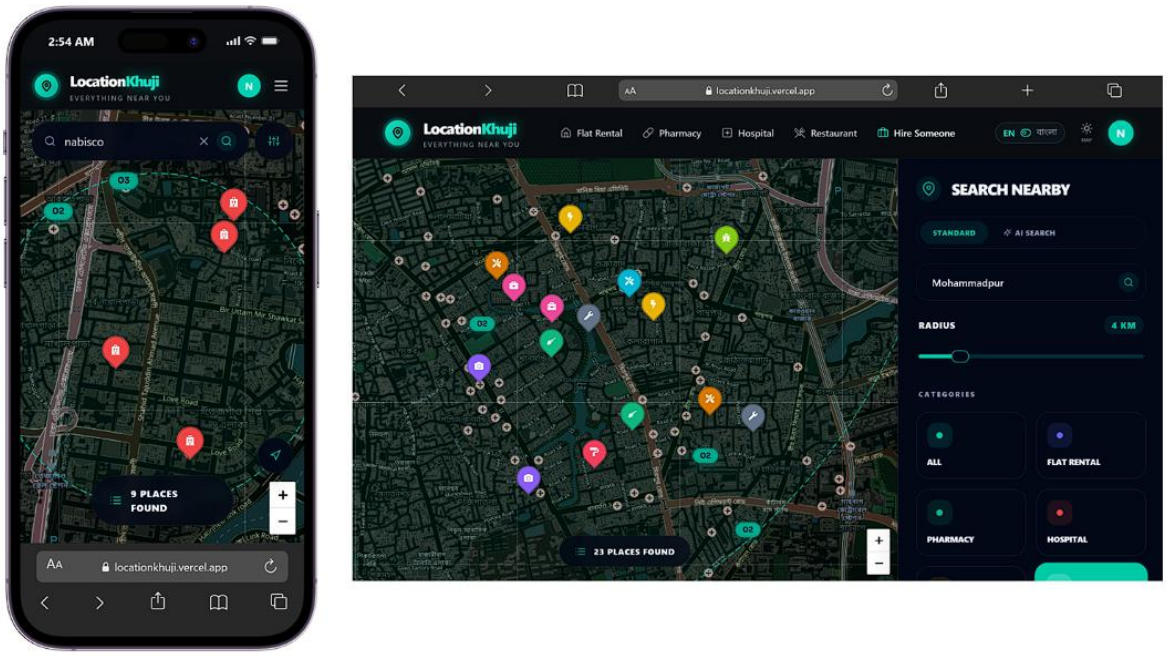

Figure 4.6: Standard search and filtering options.

7.3.7 Nearby Location Discovery

LocationKhuji uses MongoDB geospatial queries to find listings near a selected coordinate. Users can search around their current location, click on the map to choose a location, or search by area name.

Benefit: Users can discover useful places close to their current or selected location.

Screenshot:

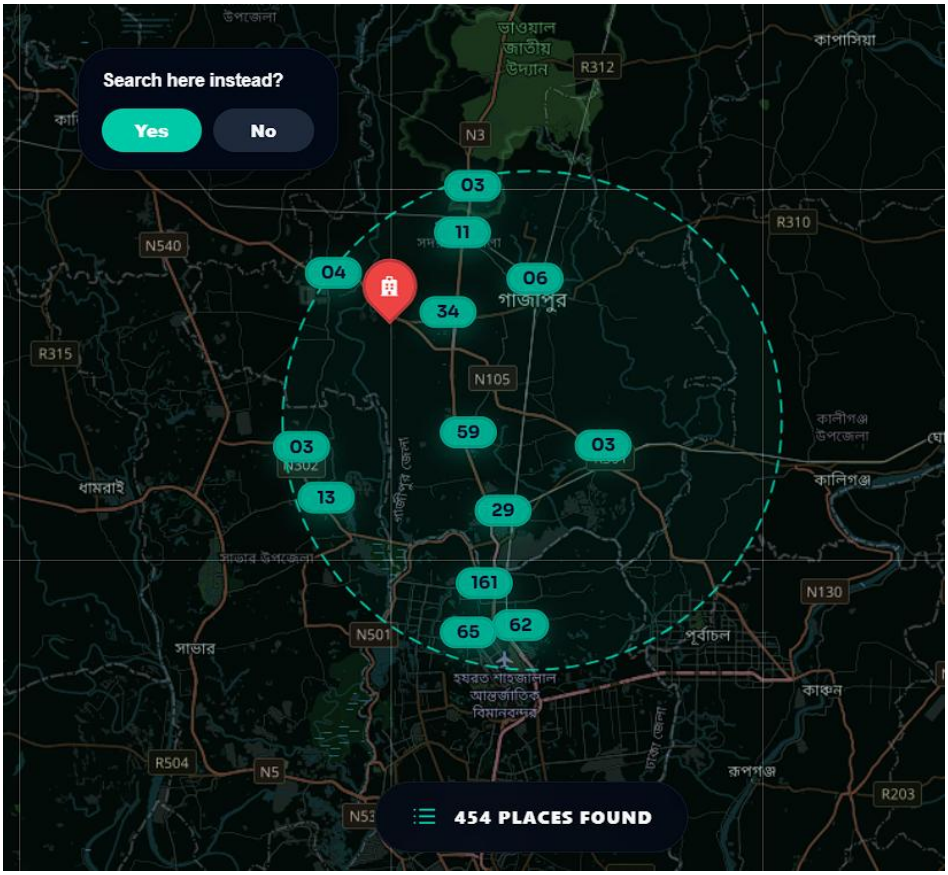

Figure 4.7: Nearby listings displayed according to selected location.

7.3.8 Search by Listing Categories

Users can select a division first, then choose a district, and finally choose a thana or upazila such as Dhaka -> Dhaka -> Savar. After selection, the listing page filters the results and updates the place count and listing cards according to the selected area.

Benefit: Users who prefer manual filtering can narrow down results accurately by area and category.

Screenshot:

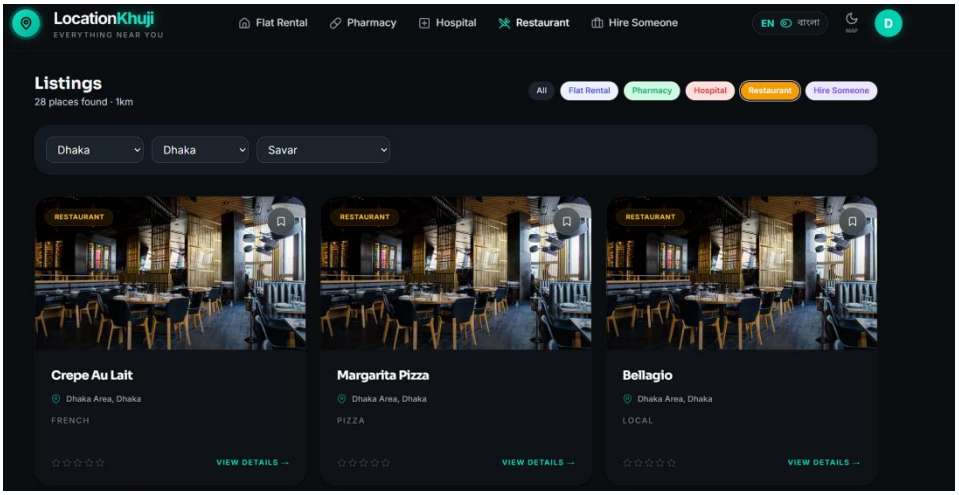

Figure 4.8: Listing categories supported by LocationKhuji.

7.3.9 Listing Creation and Management

Business owners can create, edit, and remove their own listings. The listing form includes title, description, category, address, contact details, images, and exact map coordinates. Owners can place the listing pin manually on the map or use geocoding.

Benefit: Business owners can publish accurate and useful location information without admin assistance.

Screenshot:

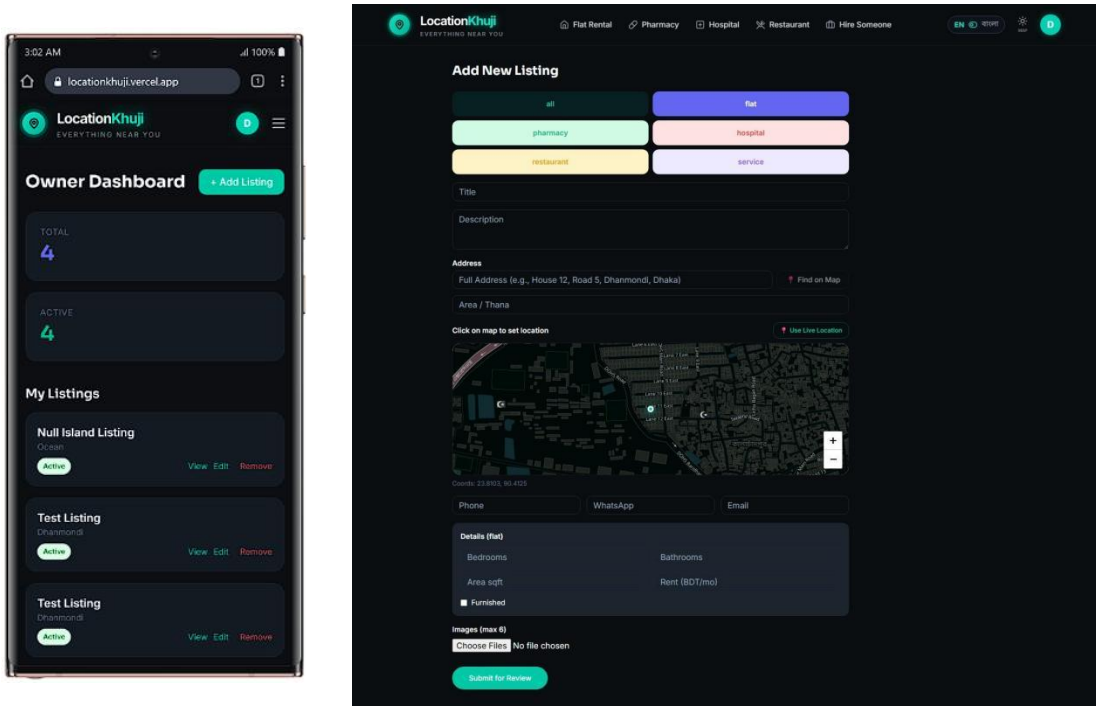

Figure 4.9: Owner listing creation form with map location selection.

7.3.10 Listing Detail Page

The listing detail page shows full information about a selected listing. It includes images, address, category details, contact information, owner information, embedded map, reviews, save option, report option, and direction link.

Benefit: Users can make a decision from one page without searching for information in different places.

Screenshot:

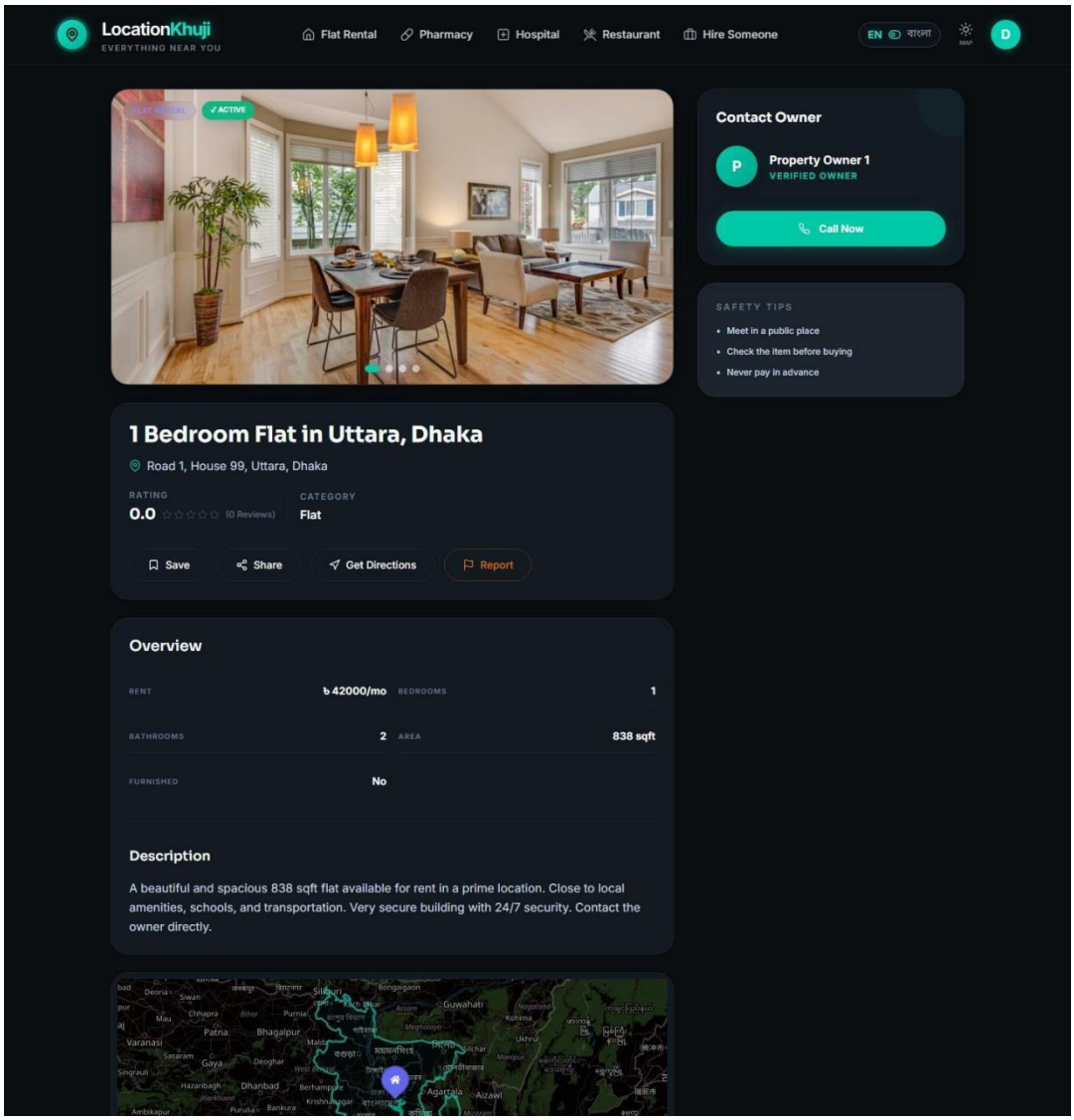

Figure 4.10: Detailed listing view with contact and map information.

7.3.11 Image Upload

Owners can upload listing images through the application. The backend validates file type and size, uploads images to Cloudinary, and stores file metadata in MongoDB.

Benefit: Images make listings more trustworthy and help users visually understand the place or service.

Screenshot:

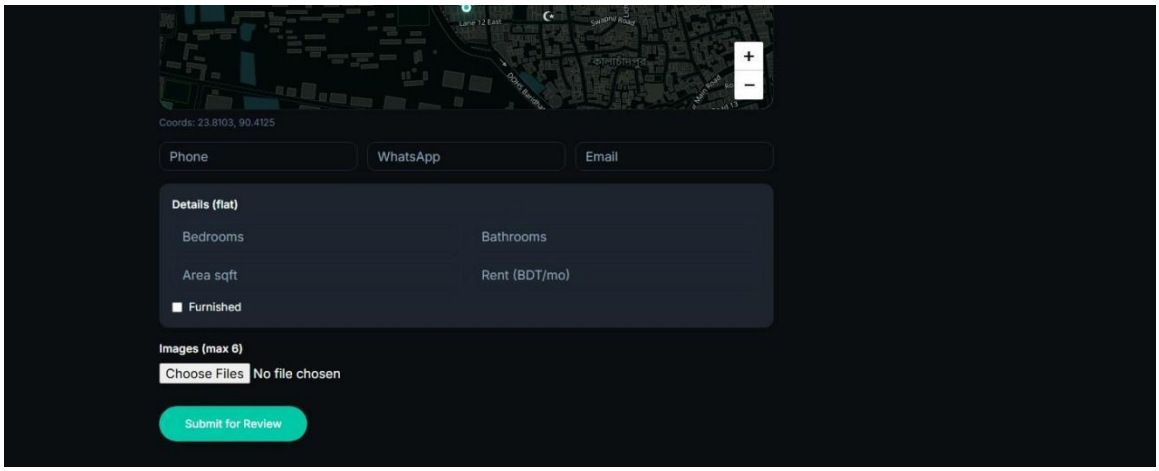

Figure 4.11: Image upload and listing gallery feature.

7.3.12 Review and Rating System

Authenticated users can submit a rating and written review for listings. The system allows one review per user per listing and automatically recalculates the average rating after a review is added or deleted.

Benefit: Reviews help users judge listing quality based on other users' experiences.

Screenshot:

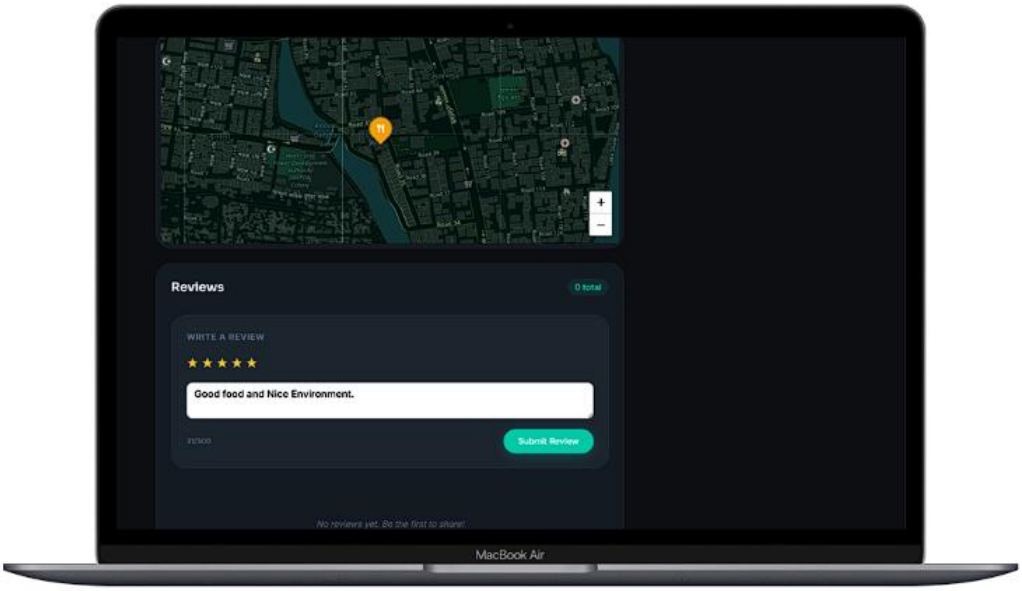

Figure 4.12: Review and rating system for listings.

7.3.13 Saved Listings

Users can save or unsave listings from listing cards or the detail page. Saved listings are stored in the user's profile and can be viewed later from the saved listings page.

Benefit: Users can bookmark important places and return to them easily.

Screenshot:

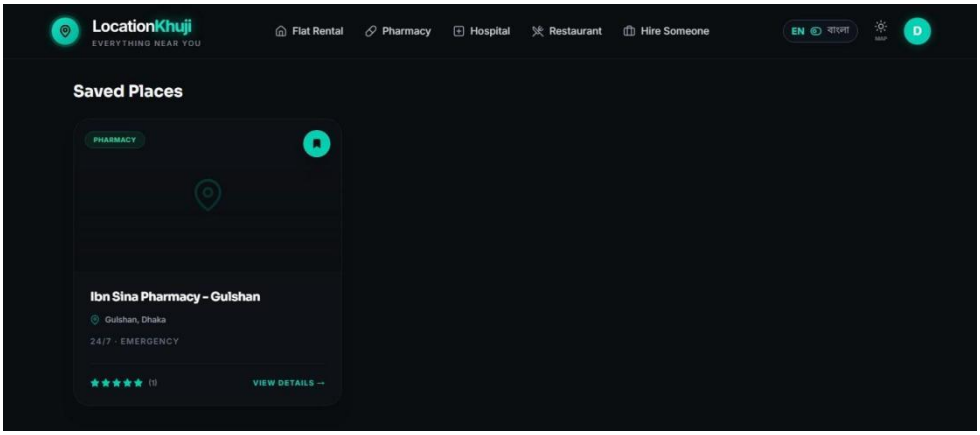

Figure 4.13: Saved listings and bookmark feature.

7.3.14 Owner Dashboard

The owner dashboard shows all listings created by the logged-in business owner. Owners can view, edit, delete, and add listings from this dashboard.

Benefit: Business owners get a simple control panel for managing their published listings.

Screenshot:

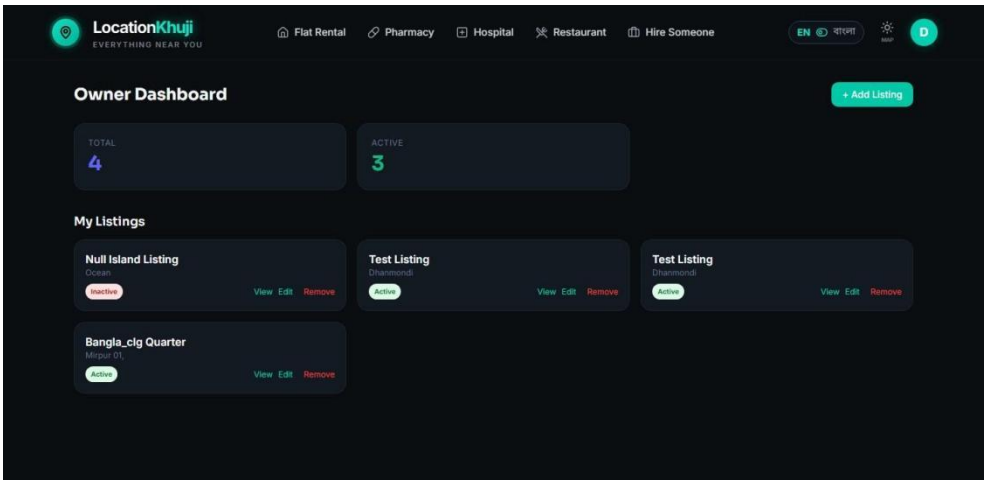

Figure 4.14: Owner dashboard for listing management.

7.3.15 Admin Dashboard

The admin dashboard provides platform-level controls. Admins can view statistics, manage users, change user roles or active status, view all listings, and mark listings as featured.

Benefit: Administrators can maintain data quality and control platform activity.

Screenshot:

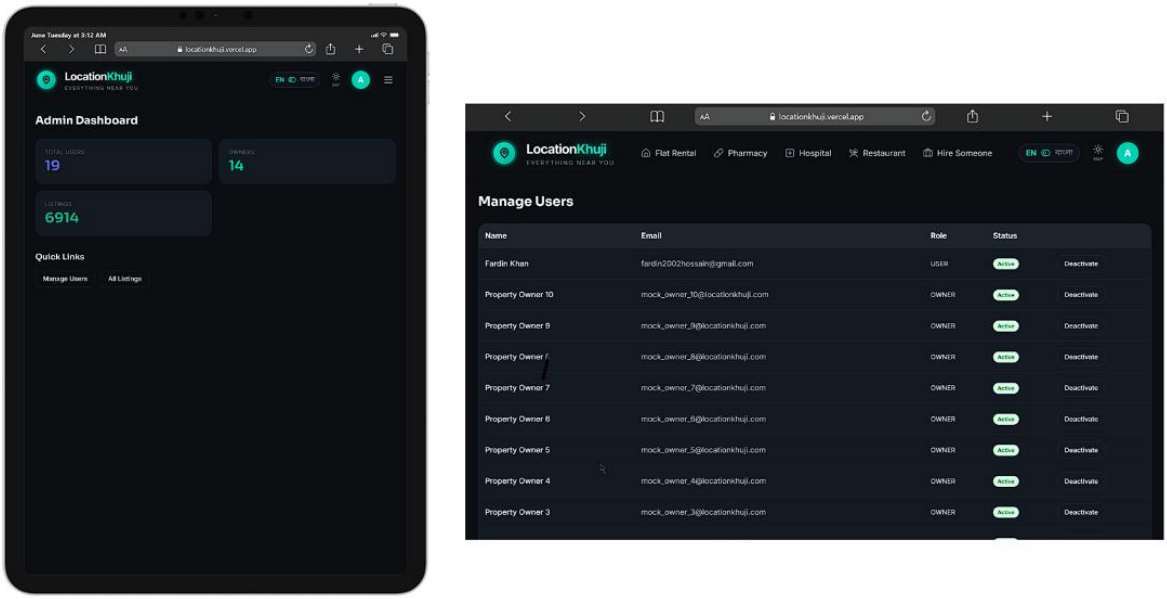

Figure 4.15: Admin dashboard and platform management features.

7.3.16 Real-Time Listing Updates

LocationKhuji uses Socket.io to broadcast new listings to connected clients. When an owner creates a listing or the system seeds a new OpenStreetMap listing, active map users can receive the new listing without refreshing the page.

Benefit: The map stays updated in real time and shows newly available listings instantly.

Screenshot:

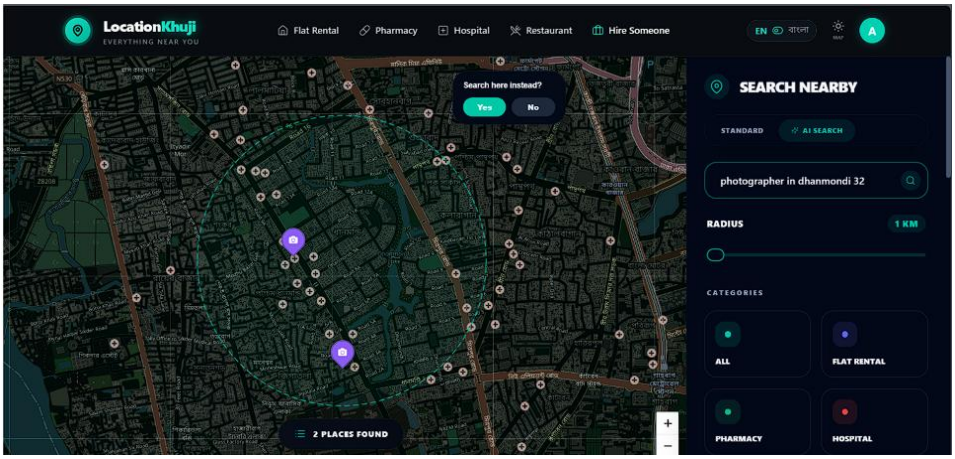

Figure 4.16: Real-time listing update on the map.

7.3.17 OpenStreetMap Live Seeding

If the local database has no results for some searches, the backend can fetch public place data from OpenStreetMap Overpass API. The system maps OSM places to LocationKhuji categories, checks for duplicates, stores new listings, and broadcasts them to users.

Benefit: The platform can provide useful results even when its own database has limited data for an area.

Screenshot:

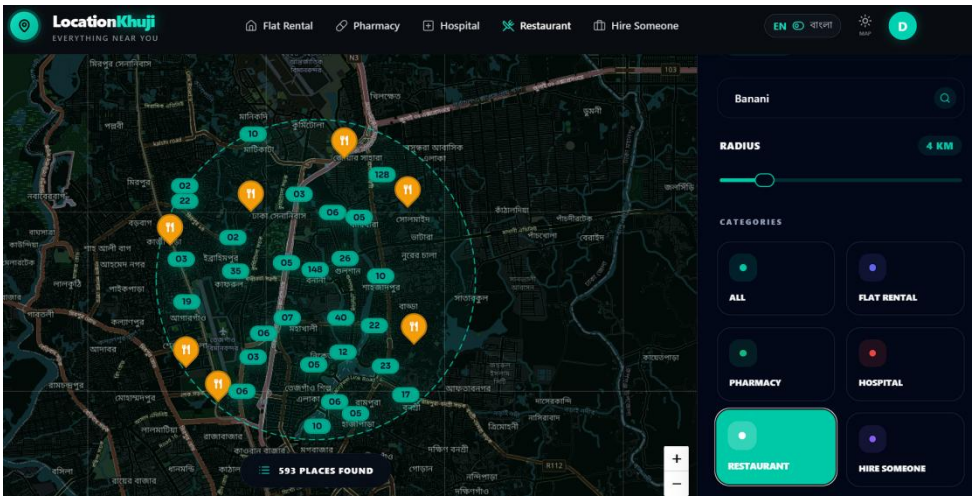

Figure 4.17: OpenStreetMap live-seeded search results.

7.3.18 Bilingual Support

The interface supports both English and Bengali using i18next. The system also normalizes Bengali digits and recognizes Bengali or Banglish category/location words in search.

Benefit: Bangladeshi users can use the application more comfortably in their preferred language.

Screenshot:

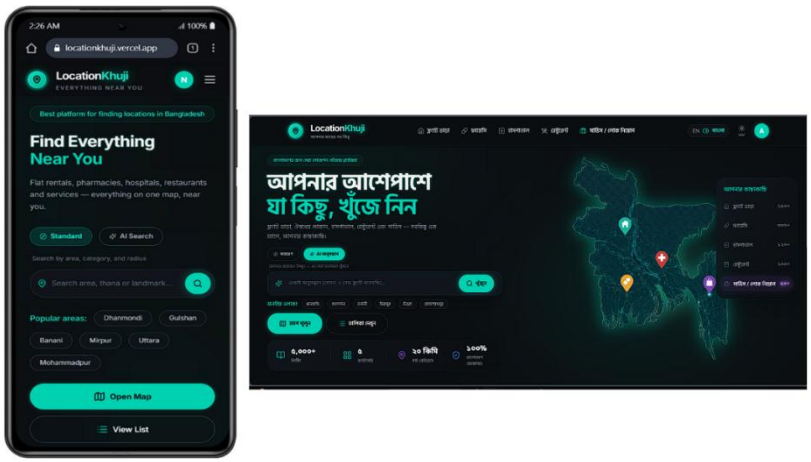

Figure 4.18: Bengali and English language support.

7.3.19 Responsive Mobile Design

The application is designed for desktop, tablet, and mobile screens. On mobile devices, the map uses a compact layout with touch-friendly controls, floating search, and drawer-style panels.

Benefit: Users can search and browse listings easily from smartphones.

Screenshot:

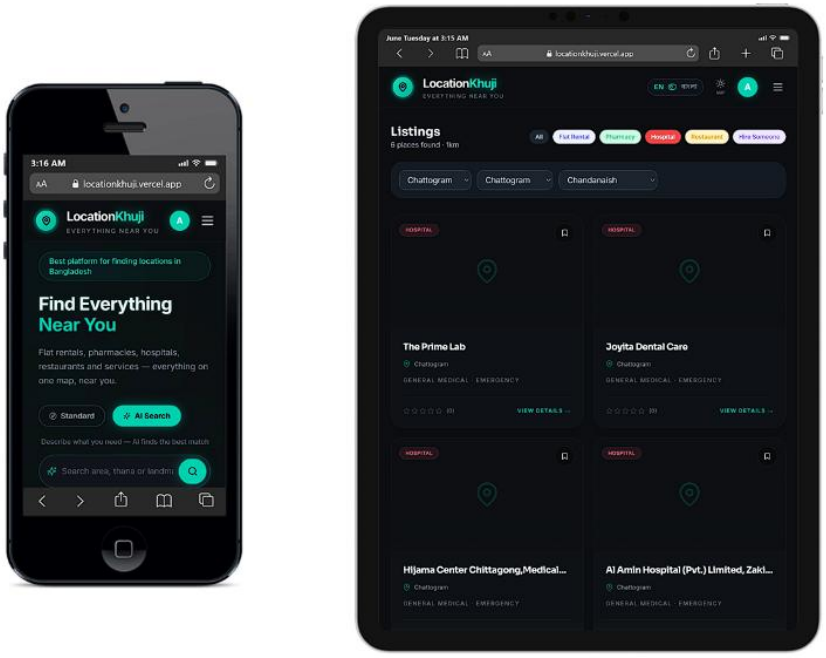

Figure 4.19: Mobile responsive interface of LocationKhuji.

7.3.20 Email Verification and Password Reset

The system sends account verification and password reset emails. Resend is used as the primary email provider, and Firebase email service works as a fallback.

Benefit: Users can verify accounts and recover access if they forget their password.

Screenshot:

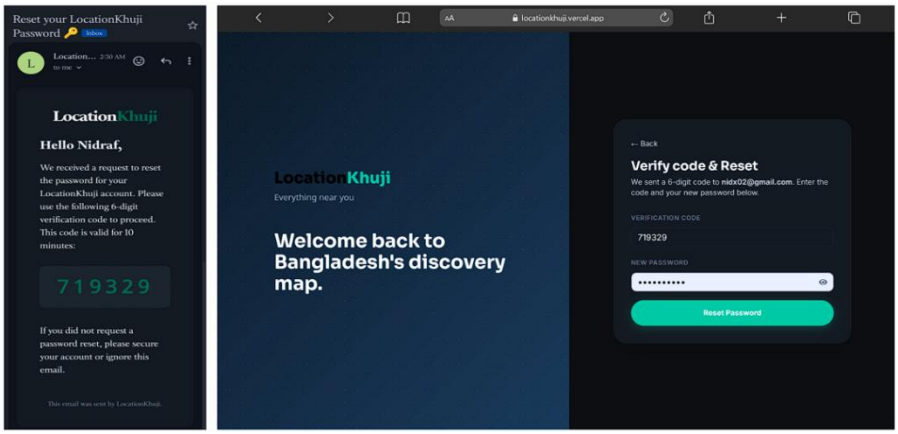

Figure 4.20: Email verification and password reset workflow.

7.3.21 Bangladesh Boundary Enforcement

LocationKhuji is restricted to Bangladesh. The backend rejects listing coordinates outside Bangladesh, and the frontend map prevents users from panning far outside the Bangladesh boundary.

Benefit: Search results and listings remain relevant to the purpose of the platform.

Screenshot:

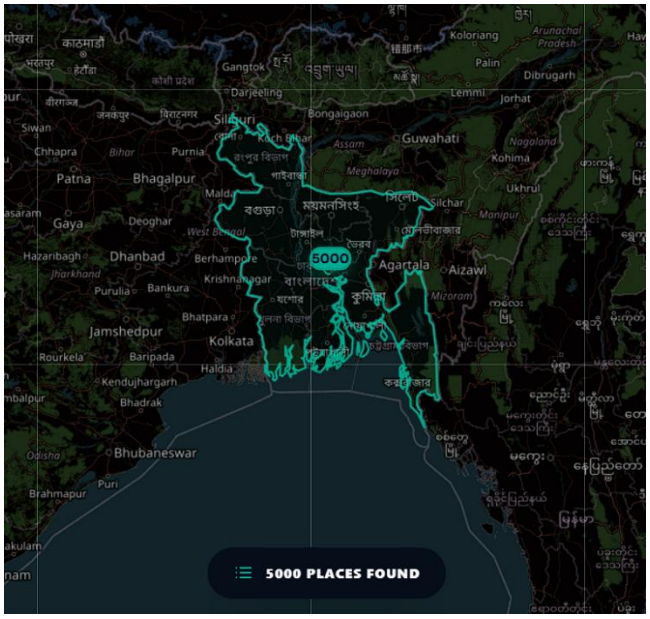

Figure 4.21: Bangladesh-only geospatial boundary enforcement.

7.3.22 Dark and Light Map Theme

The map supports dark and light visual modes. Users can switch the map theme from the navigation bar, and the preference is stored locally.

Benefit: Users can choose a map style that is comfortable for their environment and device.

Screenshot:

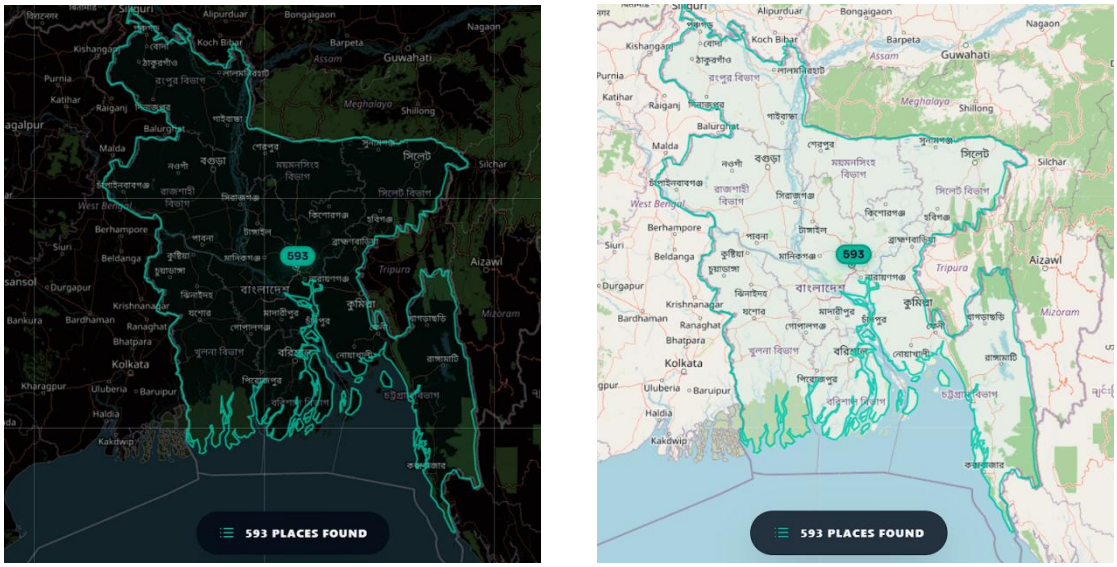

Figure 4.22: Dark and light map theme support.

8. Testing and Quality Assurance

8.1 Testing Strategy

The following table presents twelve functional test cases covering authentication, search functionality, CRUD operations, input validation, and user interface interactions.

8.2 Functional Test Cases

TC ID	Scenario	Expected Result	Status
TC-01	Register valid user.	Account created, verification email sent, MongoDB user stored.	Pass
TC-02	Login with valid credentials.	Token/cookies issued and protected pages accessible.	Pass
TC-03	Login with invalid password.	Error response and visible message.	Pass
TC-04	AI search pharmacy in Mirpur 10.	Category and location parsed; map focuses correctly.	Pass
TC-05	Bengali/Banglish query.	Digits and terms normalized correctly.	Pass
TC-06	Standard keyword/category search.	Relevant listings sorted by priority.	Pass
TC-07	Create listing inside Bangladesh.	Listing saved and broadcast.	Pass
TC-08	Coordinates outside Bangladesh.	Backend rejects request.	Pass
TC-09	Unsupported upload type.	Upload rejected.	Pass
TC-10	Submit review.	Review saved and rating recalculated.	Pass
TC-11	Save/unsave listing.	Saved list toggles.	Pass
TC-12	Admin feature toggle.	Listing featured flag updates.	Pass

8.3 Traceability Matrix

Requirement	Implementation Evidence	Test Case
FR-01	Auth routes and ProtectedRoute.	TC-01 to TC-03
FR-03/FR-04	GET /listings/search and POST /listings/ai-search.	TC-04 to TC-06
FR-05	MapPage, MapView and BangladeshMask.	TC-04, TC-05
FR-06/FR-07	Owner listing forms, /api/listings and /api/uploads.	TC-07 to TC-09
FR-08/FR-09	Saved listing and review endpoints.	TC-10, TC-11
FR-10	Admin routes and report count support.	TC-12
FR-11	Socket.io new_listing event.	TC-07

8.4 Known Quality Gaps

- Automated backend API tests should be added for auth, listing validation, search and review aggregation.
- End-to-end tests should cover registration, login, AI search, owner listing creation and admin moderation.
- Secrets found in local environment files should be rotated before public sharing or production use.
- CORS should be explicitly restricted in production.

9. Deployment and Configuration

Component	Target	Notes
Frontend SPA	Vercel	Public URL: https://locationkhuji.vercel.app/.
Backend API	Render	Node.js service exposes /api routes.
Database	MongoDB Atlas	Cloud collections and geospatial indexes.
Authentication	Firebase	Email/password auth and ID token verification.
Media	Cloudinary	Image upload and CDN delivery.
External data	OpenStreetMap	Nominatim geocoding and Overpass fallback data.
Email	Resend/Firebase	Verification and password reset workflows.

9.2 Configuration

Application	Required Variables
Backend	PORT, CORS_ORIGIN, MONGO_URL, DB_NAME, Firebase credentials, Cloudinary credentials, GROQ_API_KEY, OPENROUTER_API_KEY, RESEND_API_KEY.
Frontend	REACT_APP_BACKEND_URL and Firebase client configuration values.

Actual secret values must never be printed in the report or committed to a public repository. Rotate exposed keys before final public deployment.

9.3 Build and Run Commands

```
npm run dev:hub        # start frontend workspace

npm run dev:api        # start backend workspace

npm run build:hub      # build frontend

npm run build:all      # build all workspaces

npm run test:all       # run workspace tests if present
```

10. Maintenance Plan

Type	Planned Activities
Corrective	Fix bugs in search, map focusing, authentication, upload failures, UI layout and API errors.
Adaptive	Update integrations when Firebase, Cloudinary, Groq, OpenRouter, Nominatim, Overpass or hosting providers change.
Perfective	Improve search ranking, admin moderation, owner analytics, prompts and filters.
Preventive	Rotate secrets, upgrade dependencies, add tests, monitor logs, back up database and review security.

10.2 Routine Checklist

- Weekly: review backend logs, failed requests, OSM/AI errors and email delivery failures.
- Weekly: check MongoDB storage, indexes and slow query logs.
- Monthly: update dependencies after local testing.
- Monthly: audit Cloudinary usage and remove unused images.
- Quarterly: rotate API keys and review hosting environment variables.
- Before each release: run build, smoke-test auth/search/listing/admin flows and verify mobile layout.

10.3 Future Roadmap

Priority	Enhancement
High	Build a location-based e-service ordering system where users can request services directly from nearby listings.
High	Add pharmacy medicine ordering with prescription upload, delivery or pickup option, and order status tracking.
High	Add hospital e-service booking for doctor appointments, diagnostic tests, available schedules, and booking confirmation.
High	Add flat rental workflow for visit booking, rent request, owner confirmation, and booking status tracking.
High	Add service booking for plumbers, electricians, cleaners, tutors, photographers, event managers, and other local service providers.
High	Allow normal users to publish verified self-service listings, so individuals can offer skills or small services after moderation.

Medium	Add provider dashboards to manage orders, bookings, availability, customer requests, and service status updates.
Medium	Add notifications, order history, invoices, and optional online payment integration for completed e-service workflows.
Medium	Develop a React Native mobile app after the web-based e-service workflows become stable.

11. Risk Management

Risk	Impact	Mitigation
External API quota	AI search, geocoding or OSM fallback may fail.	Use cache, fallback models and background fetching.
Secret exposure	Unauthorized access to cloud services or database.	Never publish .env values, rotate keys and use hosting secrets.
Incorrect location matching	Wrong map focus or irrelevant listings.	Use local aliases, BD engine, confidence scoring and strict unresolved-location behavior.
Database growth	Search/dashboard performance degradation.	Maintain indexes, paginate results and monitor slow queries.
Security misconfiguration	Unauthorized API access.	Review CORS, Firebase claims and role middleware regularly.
Dependency vulnerabilities	Build or runtime security issues.	Run audits, update dependencies and test after upgrades.

12. Validation & Security

LocationKhuji implements multiple layers of security and input validation to protect against common web application vulnerabilities and ensure data integrity.

12.1 Input Validation

All search inputs are sanitized by escaping regex special characters before constructing MongoDB queries. This prevents NoSQL injection attacks and Regular Expression Denial of Service (ReDoS) exploits that could degrade server performance.

12.2 Authentication & Authorization

Firebase Admin SDK verifies JWT ID tokens on every protected endpoint. Role-based middleware functions (requireAuth and requireRoles) guard API routes. Access and refresh tokens are stored in HTTP-only cookies, preventing cross-site scripting (XSS) token theft.

12.3 Environment Variable Protection

All sensitive API keys — including Firebase, Cloudinary, Groq, OpenRouter, Resend, and MongoDB connection URIs — are stored exclusively in .env files. The .gitignore configuration blocks all environment files from version control.

12.4 CORS Configuration

A configurable CORS_ORIGIN environment variable restricts API access to authorised frontend domains in production, preventing unauthorised cross-origin requests.

12.5 File Upload Sanitization

Multer enforces a 5 MB file size limit and validates both file extension and MIME type, accepting only image formats. This prevents malicious file uploads and server resource exhaustion.

12.6 Geospatial Boundary Enforcement

Server-side validation rejects any listing coordinates outside Bangladesh's bounding box (latitude 20.3°–26.7°, longitude 88.0°–92.7°), ensuring geographic data integrity.

12.7 Rate Limiting on External APIs

In-memory caching layers — AI Intent Cache (1,000 entries), Geocode Cache (1,000 entries), and OSM Cache (500 entries) — prevent excessive calls to external API services and reduce response latency.

13. Conclusion and Reflection

This was my first large-scale full-stack project, and the development journey was both extremely challenging and highly rewarding. Through building LocationKhuji, I gained practical experience in designing and implementing a complete modern web application using React, Express.js, MongoDB, Socket.io, AI APIs, and real-time map systems.

One of the biggest challenges I faced throughout the project was ensuring accurate location detection and map focusing. Since the platform heavily depends on geospatial searching and intelligent location matching, small mistakes often created major issues. For example, locations would sometimes focus on the wrong area, markers would appear outside the intended region, or previously searched locations would persist incorrectly on the map. I spent a significant amount of time debugging coordinate systems, radius filtering, stale frontend states, and MongoDB geospatial queries.

Another major challenge was that fixing one issue often created another unexpected issue somewhere else in the application. In many cases, I successfully fixed a bug, but after adding a new feature or modifying the search pipeline, the same bug would reappear in a different form. This repeatedly happened during the AI search implementation, map synchronization system, and responsive UI development. At times, this became frustrating, but it also taught me an important lesson about software engineering: large systems require structured planning, modular thinking, and proper debugging workflows.

While facing these recurring problems, I discovered the concept of **Spec-Driven Development** and learned the **Six-File Methodology** from the following YouTube resource:

JavaScript Mastery. (2026) *Six-File Methodology for Spec-Driven Development*. Available at: Six-File Methodology Video (Accessed: 30 May 2026).

Learning this methodology significantly improved my development workflow. Instead of randomly fixing problems, I started organizing features more systematically by separating planning, architecture, state handling, backend logic, UI behavior, and testing into structured development stages. This helped me better manage complex features and reduce repeated bugs.

Implementing the AI-powered search pipeline was another major learning experience. I worked with multiple AI systems including Groq Llama 3, OpenRouter DeepSeek, and a regex-based fallback engine to ensure the search system remained functional even if one AI provider failed. Managing fallback behavior, API errors, and multilingual search matching taught me practical concepts related to resilient system design.

The project also improved my understanding of responsive frontend development. Making the interface fully responsive across mobile, tablet, and desktop devices required extensive testing and UI optimization. I learned how difficult it can be to maintain consistent layouts for maps, filters, search panels, and dynamic listing components across different screen sizes.

Overall, this project greatly improved my problem-solving ability, debugging skills, system design understanding, and confidence in independent software development. If I continue developing this project in the future, I would focus more on creating smaller modular backend services, improving automated testing, and optimizing the overall architecture for better scalability and maintainability.

14. References

1. SimpleMaps (2024) Bangladesh GIS and map data. Available at: https://simplemaps.com (Accessed: 10 May 2026).
2. Montasim, M. (2024) address-bd: Bangladesh address dataset for divisions, districts and upazilas. GitHub repository. Available at: https://github.com/montasim/address-bd (Accessed: 12 May 2026).
3. Google AI Pro and Google AI Studio. Available at: https://aistudio.google.com (Accessed: 18 May 2026).
4. JavaScript Mastery (2026) JavaScript Mastery YouTube Channel. Available at: https://www.youtube.com/@javascriptmastery (Accessed: 21 May 2026).
5. Nano Banana AI (2026) Nano Banana AI Platform. Available at: https://nanobanana.ai (Accessed: 27 May 2026).
6. OpenAI (2026) OpenAI Codex. Available at: https://openai.com (Accessed: 1 June 2026).
7. React (2024) React Documentation. Available at: https://react.dev (Accessed: 5 May 2026).
8. MongoDB (2024) MongoDB Documentation. Available at: https://www.mongodb.com/docs (Accessed: 8 May 2026).
9. Leaflet (2024) Leaflet — an open-source JavaScript library for interactive maps. Available at: https://leafletjs.com (Accessed: 10 May 2026).
10. Firebase (2024) Firebase Documentation. Available at: https://firebase.google.com/docs (Accessed: 12 May 2026).